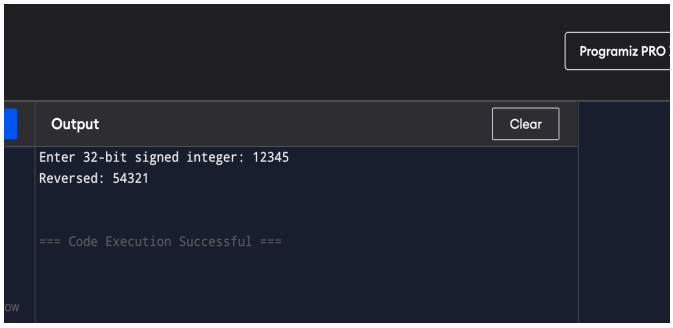
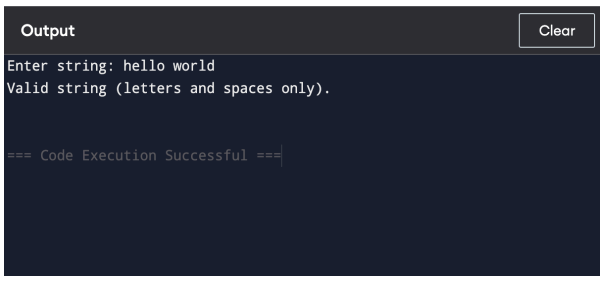
Q1 — Reverse a 32-bit signed integer (overflow-safe).

```
#include <stdio.h>
#include <limits.h>

int reverse_int(int x) {
    long long rev = 0;
    while (x != 0) {
        int digit = x % 10;
        rev = rev * 10 + digit;
        if (rev > INT_MAX || rev < INT_MIN) return 0; // overflow
        x /= 10;
    }
    return (int)rev;
}

int main() {
    int n;
    printf("Enter 32-bit signed integer: ");
    if (scanf("%d", &n) != 1) return 0;
    int r = reverse_int(n);
    if (r == 0) printf("Reversed value is 0 (possible overflow or original reversed to 0).\n");
    else printf("Reversed: %d\n", r);
    return 0;
}
```



Q2 — Check for a valid string

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int is_valid_string(const char *s) {
    for (int i = 0; s[i]; ++i) {
        if (!(isalpha((unsigned char)s[i]) || isspace((unsigned char)s[i]))) return 0;
    }
    return 1;
}
int main() {
    char buf[1024];
    printf("Enter string: ");
    if (!fgets(buf, sizeof(buf), stdin)) return 0;
    // remove newline
    buf[strcspn(buf, "\n")] = 0;
    if (is_valid_string(buf)) printf("Valid string (letters and spaces only).\n");
    else printf("Invalid string (contains non-letters/non-space characters).\n");
    return 0;
}
```



Q3) WRITE REG NO USING BINARY AND LINEAR SEARCH

```
#include <stdio.h>
int main() {
```

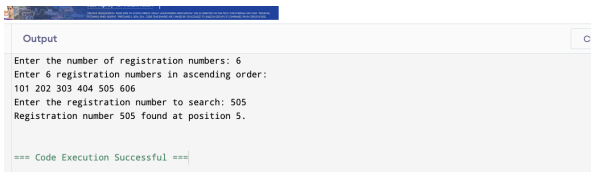
```
int arr[100], n, key, i, found = 0;

// Input number of registration numbers
printf("Enter the number of registration numbers: ");
scanf("%d", &n);

// Input registration numbers
printf("Enter %d registration numbers:\n", n);
for (i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}
printf("Enter the registration number to search: ");
scanf("%d", &key);

// Linear Search
for (i = 0; i < n; i++) {
    if (arr[i] == key) {
        found = 1;
        break;
    }
}

// Display result
if (found)
    printf("Registration number %d found at position %d.\n", key, i + 1);
else
    printf("Registration number %d not found.\n", key);

return 0;
```

}

BINARY SEARCH

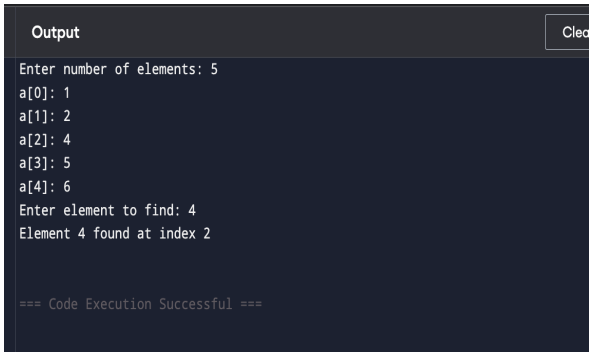
```
#include <stdio.h>
int main() {
    int arr[100], n, key, low, high, mid, found = 0;

    // Input number of registration numbers
    printf("Enter the number of registration numbers: ");
    scanf("%d", &n);

    // Input registration numbers (sorted order)
    printf("Enter %d registration numbers in ascending order:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Input the registration number to search
    printf("Enter the registration number to search: ");
    scanf("%d", &key);

    // Binary Search
    low = 0;
    high = n - 1;
```

```
    while (low <= high) {
        mid = (low + high) / 2;

        if (arr[mid] == key) {
            found = 1;
            break;
        } else if (arr[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    // Display result
    if (found)
        printf("Registration number %d found at position %d.\n", key, mid + 1);
    else
        printf("Registration number %d not found.\n", key);

    return 0;
}
```



Q4— Identify location (index) of element in given array

```
#include <stdio.h>


int main() {
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 0;
    int a[n];
    for (int i = 0; i < n; ++i) {
        printf("a[%d]: ", i);
        scanf("%d", &a[i]);
    }
    int key;
    printf("Enter element to find: ");
    scanf("%d", &key);
    int found = 0;
    for (int i = 0; i < n; ++i) {
        if (a[i] == key) {
            printf("Element %d found at index %d\n", key, i);
            found = 1;
            break; // remove break to print all occurrences
        }
    }
    if (!found) printf("Element not found\n");
    return 0;
}
```



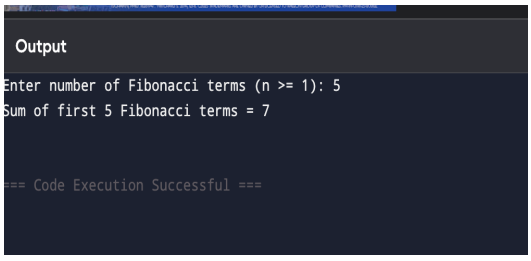
Q5— Given array print odd and even values

```
#include <stdio.h>

int main() {
    int n;
    printf("Number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 0;
    int a[n];
    for (int i = 0; i < n; ++i) scanf("%d", &a[i]);

    printf("Even numbers: ");
    for (int i = 0; i < n; ++i) if (a[i] % 2 == 0) printf("%d ", a[i]);
    printf("\nOdd numbers: ");
    for (int i = 0; i < n; ++i) if (a[i] % 2 != 0) printf("%d ", a[i]);
    printf("\n");
    return 0;
```


```
}
```

Q6 — Sum of Fibonacci series (sum of first n Fibonacci numbers)

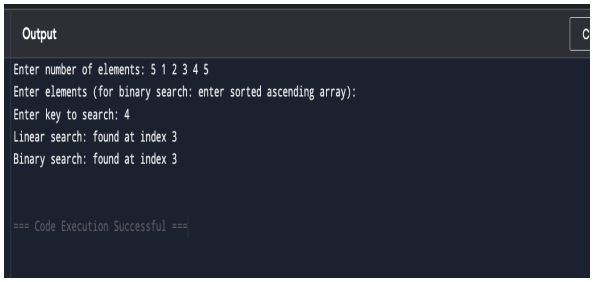
```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of Fibonacci terms (n >= 1): ");
    if (scanf("%d", &n) != 1 || n < 1) return 0;
    long long a = 0, b = 1;
    long long sum = 0;
    if (n >= 1) sum += a; // include F0 if desired; if you want starting at F1 change accordingly
    if (n >= 2) sum += b;
    for (int i = 3; i <= n; ++i) {
        long long c = a + b;
        sum += c;
        a = b; b = c;
    }
    printf("Sum of first %d Fibonacci terms = %lld\n", n, sum);
    return 0;
}
```



Q7 — Factorial of a number

```
#include <stdio.h>
```

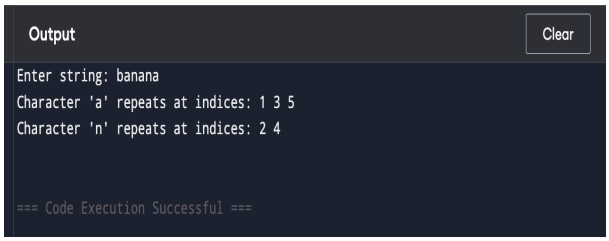
```
unsigned long long factorial(unsigned int n) {
    unsigned long long res = 1;
    for (unsigned int i = 2; i <= n; ++i) res *= i;
    return res;
}

int main() {
    unsigned int n;
    printf("Enter non-negative integer: ");
    if (scanf("%u", &n) != 1) return 0;
    printf("%u! = %llu\n", n, factorial(n));
    return 0;
}
```



Q8— Print the index of repeated characters given in an array

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[1024];
    printf("Enter string: ");
    if (!fgets(s, sizeof(s), stdin)) return 0;
    s[strcspn(s, "\n")] = 0;
```

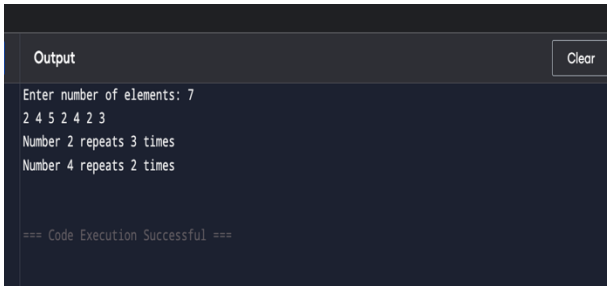
```
    int len = strlen(s);
    int printed[256] = {0};

    for (int ch = 0; ch < 256; ++ch) {
        int count = 0;
        for (int i = 0; i < len; ++i) if ((unsigned char)s[i] == ch) ++count;
        if (count > 1) {
            printf("Character '%c' repeats at indices: ", (char)ch);
            for (int i = 0; i < len; ++i) {
                if ((unsigned char)s[i] == ch) printf("%d ", i);
            }
            printf("\n");
        }
    }
    return 0;
}
```



Q9 — Print the frequently repeated numbers count from an array

```
#include <stdio.h>
#include <stdlib.h>

int cmp_int(const void *a, const void *b) {
    return (*(int*)a) - (*(int*)b);
}

int main() {
```

```
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 0;
    int *a = malloc(n * sizeof(int));
    for (int i = 0; i < n; ++i) scanf("%d", &a[i]);

    qsort(a, n, sizeof(int), cmp_int);

    int i = 0;
    int found_any = 0;
    while (i < n) {
        int j = i + 1;
        while (j < n && a[j] == a[i]) ++j;
        int count = j - i;
        if (count > 1) {
            printf("Number %d repeats %d times\n", a[i], count);
            found_any = 1;
        }
        i = j;
    }
    if (!found_any) printf("No repeated numbers\n");
    free(a);
    return 0;
}
```



Q10 — Sum of rows and columns in a 2D array

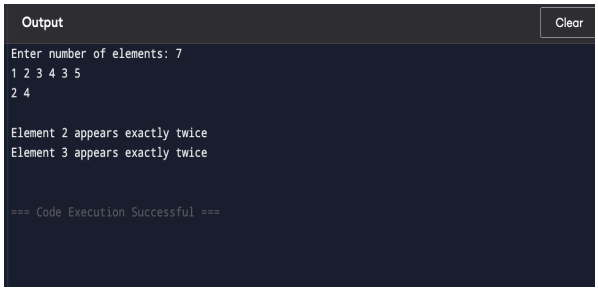
```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int r, c;
    printf("Enter rows and columns: ");
    if (scanf("%d %d", &r, &c) != 2 || r <= 0 || c <= 0) return 0;
    int **mat = malloc(r * sizeof(int*));
    for (int i = 0; i < r; ++i) mat[i] = malloc(c * sizeof(int));
    printf("Enter matrix elements row-wise:\n");
    for (int i = 0; i < r; ++i)
        for (int j = 0; j < c; ++j)
            scanf("%d", &mat[i][j]);

    for (int i = 0; i < r; ++i) {
        long long sum = 0;
        for (int j = 0; j < c; ++j) sum += mat[i][j];
        printf("Sum of row %d = %lld\n", i, sum);
    }

    for (int j = 0; j < c; ++j) {
        long long sum = 0;
```

```
        for (int i = 0; i < r; ++i) sum += mat[i][j];
        printf("Sum of column %d = %lld\n", j, sum);
    }

    for (int i = 0; i < r; ++i) free(mat[i]);
    free(mat);
    return 0;
}
```

Q11 — Check for elements repeated twice in an array

```
#include <stdio.h>
#include <stdlib.h>

int cmp_int(const void *a, const void *b) { return (*(int*)a) - (*(int*)b); }

int main() {
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 0;
    int *a = malloc(n * sizeof(int));
    for (int i = 0; i < n; ++i) scanf("%d", &a[i]);
    qsort(a, n, sizeof(int), cmp_int);
```

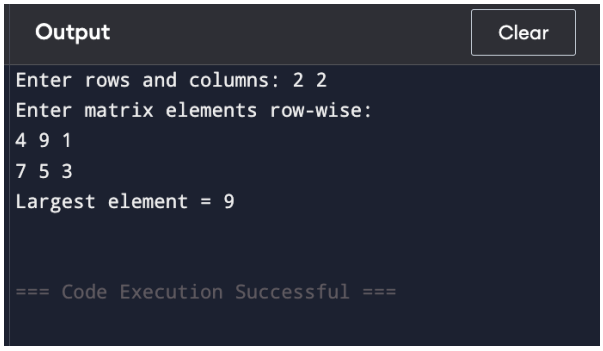
```
    int i = 0;
    int found = 0;
    while (i < n) {
        int j = i + 1;
        while (j < n && a[j] == a[i]) ++j;
        int count = j - i;
        if (count == 2) {
            printf("Element %d appears exactly twice\n", a[i]);
            found = 1;
        }
        i = j;
    }
    if (!found) printf("No element appears exactly twice\n");
    free(a);
    return 0;
}
```



Q12 — Given 2D matrix print largest element

```
#include <stdio.h>
#include <limits.h>
#include <stdlib.h>

int main() {
    int r, c;
```

```
    printf("Enter rows and columns: ");
    if (scanf("%d %d", &r, &c) != 2 || r <= 0 || c <= 0) return 0;
    int maxv = INT_MIN;
    printf("Enter matrix elements row-wise:\n");
    for (int i = 0; i < r; ++i)
        for (int j = 0; j < c; ++j) {
            int v;
            scanf("%d", &v);
            if (v > maxv) maxv = v;
        }
    printf("Largest element = %d\n", maxv);
    return 0;
}
```



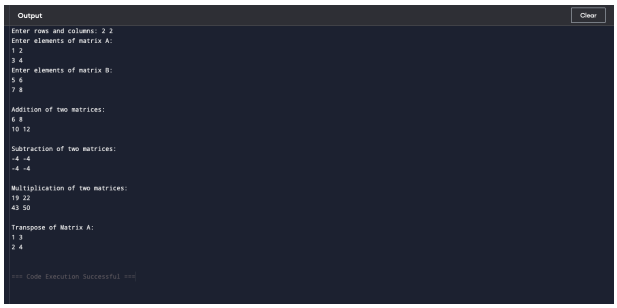
Q13. 2D add,sub,multiply,transverse

```
#include <stdio.h>

int main() {
    int A[10][10], B[10][10], C[10][10], T[10][10];
    int r, c, i, j, k;

    printf("Enter rows and columns: ");
    scanf("%d %d", &r, &c);
```

```
printf("Enter elements of matrix A:\n");
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++)
        scanf("%d", &A[i][j]);

printf("Enter elements of matrix B:\n");
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++)
        scanf("%d", &B[i][j]);

// Addition
printf("\nAddition of two matrices:\n");
for (i = 0; i < r; i++) {
    for (j = 0; j < c; j++) {
        C[i][j] = A[i][j] + B[i][j];
        printf("%d ", C[i][j]);
    }
    printf("\n");
}

// Subtraction
printf("\nSubtraction of two matrices:\n");
for (i = 0; i < r; i++) {
    for (j = 0; j < c; j++) {
        C[i][j] = A[i][j] - B[i][j];
        printf("%d ", C[i][j]);
    }
    printf("\n");
}
```

```
// Multiplication
printf("\nMultiplication of two matrices:\n");
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++) {
        C[i][j] = 0;
        for (k = 0; k < c; k++)
            C[i][j] += A[i][k] * B[k][j];
    }

for (i = 0; i < r; i++) {
    for (j = 0; j < c; j++)
        printf("%d ", C[i][j]);
    printf("\n");
}

// Transpose
printf("\nTranspose of Matrix A:\n");
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++)
        T[j][i] = A[i][j];

for (i = 0; i < c; i++) {
    for (j = 0; j < r; j++)
        printf("%d ", T[i][j]);
    printf("\n");
}

return 0;
```

}



15) 3D add,sub,multiply,transverse

```
#include <stdio.h>
int main() {
    int x, y, z;
    int A[10][10][10], B[10][10][10], C[10][10][10];
    int i, j, k;

    printf("Enter dimensions (x y z): ");
    scanf("%d %d %d", &x, &y, &z);

    printf("Enter elements of 3D Matrix A:\n");
    for (i = 0; i < x; i++)
        for (j = 0; j < y; j++)
            for (k = 0; k < z; k++)
                scanf("%d", &A[i][j][k]);

    printf("Enter elements of 3D Matrix B:\n");
    for (i = 0; i < x; i++)
        for (j = 0; j < y; j++)
            for (k = 0; k < z; k++)
                scanf("%d", &B[i][j][k]);
```

```
// Addition
printf("\n3D Matrix Addition Result:\n");
for (i = 0; i < x; i++) {
    printf("Layer %d:\n", i + 1);
    for (j = 0; j < y; j++) {
        for (k = 0; k < z; k++) {
            C[i][j][k] = A[i][j][k] + B[i][j][k];
            printf("%d ", C[i][j][k]);
        }
        printf("\n");
    }
}

// Subtraction
printf("\n3D Matrix Subtraction Result:\n");
for (i = 0; i < x; i++) {
    printf("Layer %d:\n", i + 1);
    for (j = 0; j < y; j++) {
        for (k = 0; k < z; k++) {
            C[i][j][k] = A[i][j][k] - B[i][j][k];
            printf("%d ", C[i][j][k]);
        }
        printf("\n");
    }
}

// Multiplication (layer-wise)
printf("\n3D Matrix Multiplication Result (layer-wise):\n");
for (i = 0; i < x; i++) {
```

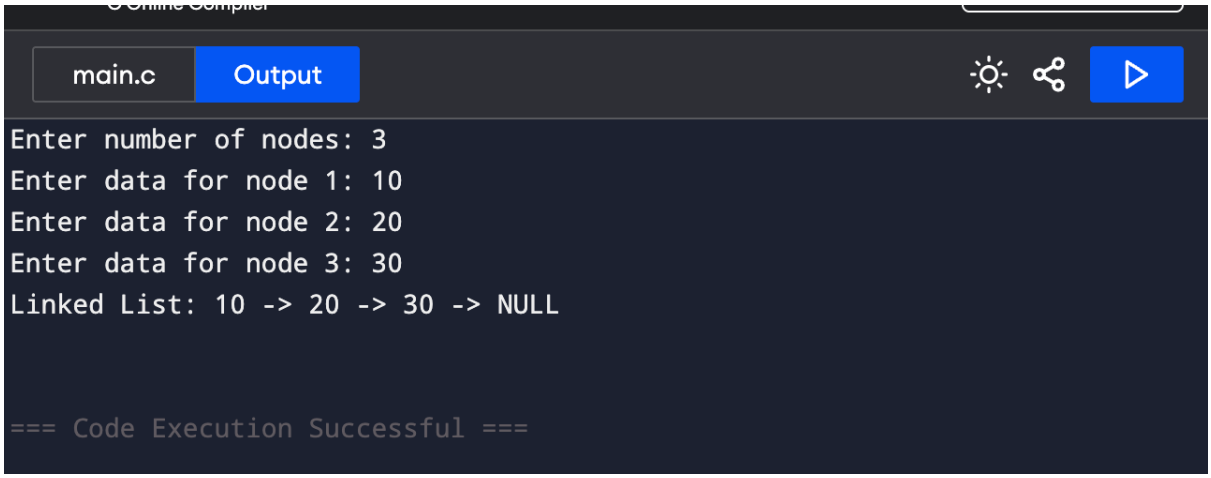
```
        printf("Layer %d:\n", i + 1);
        for (j = 0; j < y; j++) {
            for (k = 0; k < z; k++) {
                C[i][j][k] = A[i][j][k] * B[i][j][k];
                printf("%d ", C[i][j][k]);
            }
            printf("\n");
        }
    }

    return 0;
}
```

EXP NO 16: C PROGRAM TO IMPLEMENT A SINGLE LINKED LIST

```
// EXP16_SingleLinkedList.c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
```

```
int main() {
    struct Node *head = NULL, *temp, *newnode;
    int n, value;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        newnode = (struct Node *)malloc(sizeof(struct Node));
        printf("Enter data for node %d: ", i + 1);
        scanf("%d", &value);
        newnode->data = value;
        newnode->next = NULL;
        if (head == NULL)
            head = newnode;
        else {
            temp = head;
            while (temp->next)
                temp = temp->next;
            temp->next = newnode;
        }
    }

    printf("Linked List: ");
    temp = head;
    while (temp) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
    return 0;
```

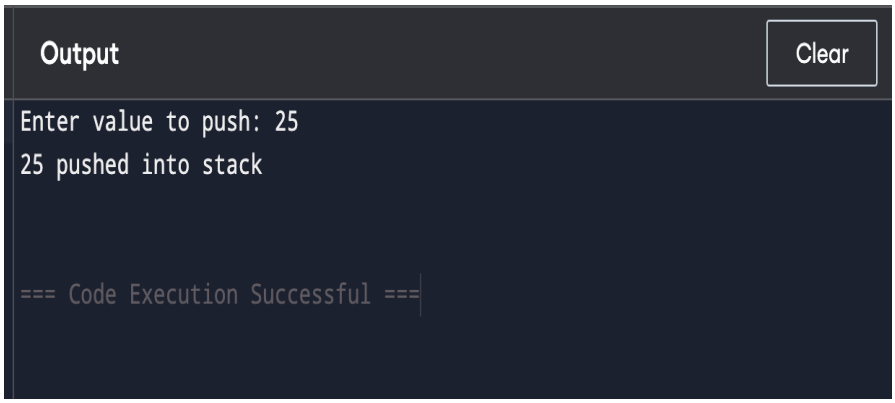
}



EXP NO17: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN ARRAY[PUSH]

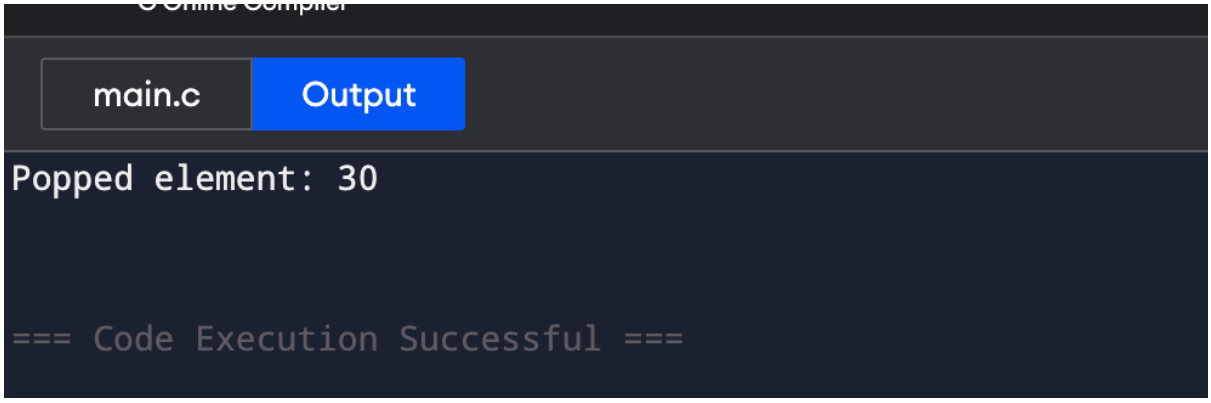
```
#include <stdio.h>

#define MAX 5

int stack[MAX], top = -1;


void push(int val) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else {
        top++;
        stack[top] = val;
        printf("%d pushed into stack\n", val);
    }
}


int main() {
    int val;
    printf("Enter value to push: ");
    scanf("%d", &val);
```

```
    push(val);
    return 0;
}
```



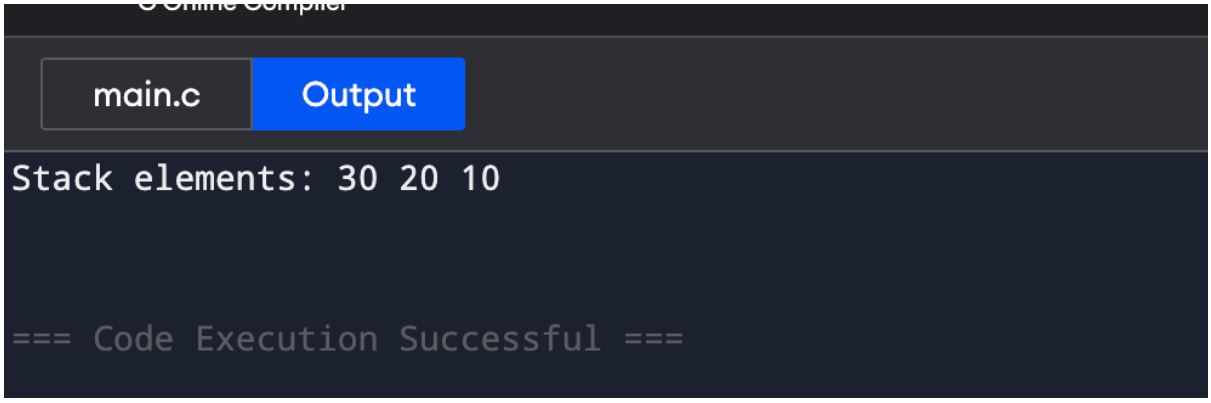
EXP NO18: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATIONIN ARRAY[POP]

```
#include <stdio.h>
#define MAX 5
int stack[MAX] = {10, 20, 30}, top = 2;

void pop() {
    if (top == -1)
        printf("Stack Underflow\n");
    else
        printf("Popped element: %d\n", stack[top--]);
}

int main() {
    pop();
    return 0;
}
```



EXP NO19: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN ARRAY[DISPLAY]

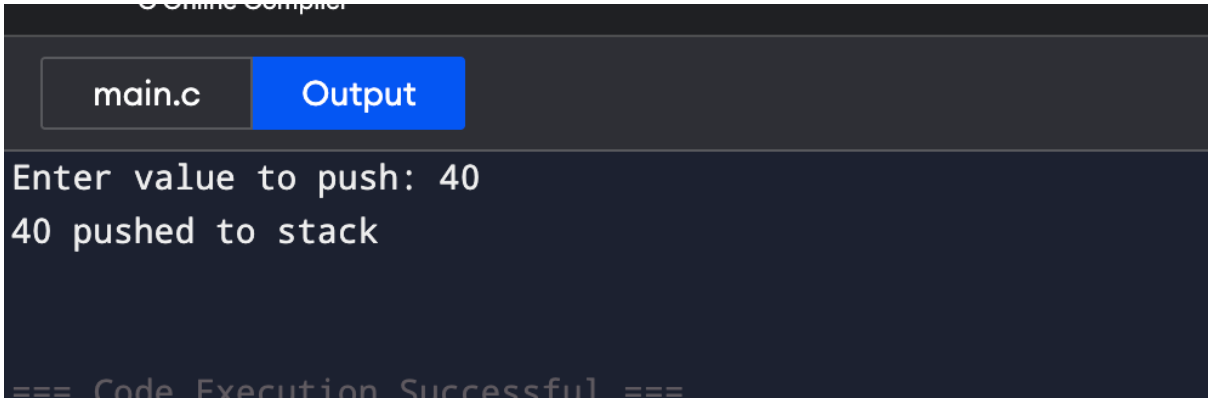
```
#include <stdio.h>

#define MAX 5

int stack[MAX] = {10, 20, 30}, top = 2;


void display() {
    if (top == -1)
        printf("Stack is empty\n");
    else {
        printf("Stack elements: ");
        for (int i = top; i >= 0; i--)
            printf("%d ", stack[i]);
        printf("\n");
    }
}


int main() {
    display();
    return 0;
}
```



EXP NO20: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN LINKED LIST[PUSH]

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int val) {
    struct Node *newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = top;
    top = newnode;
    printf("%d pushed to stack\n", val);
}

int main() {
    int val;
    printf("Enter value to push: ");
```

```
    scanf("%d", &val);
    push(val);
    return 0;
}
```

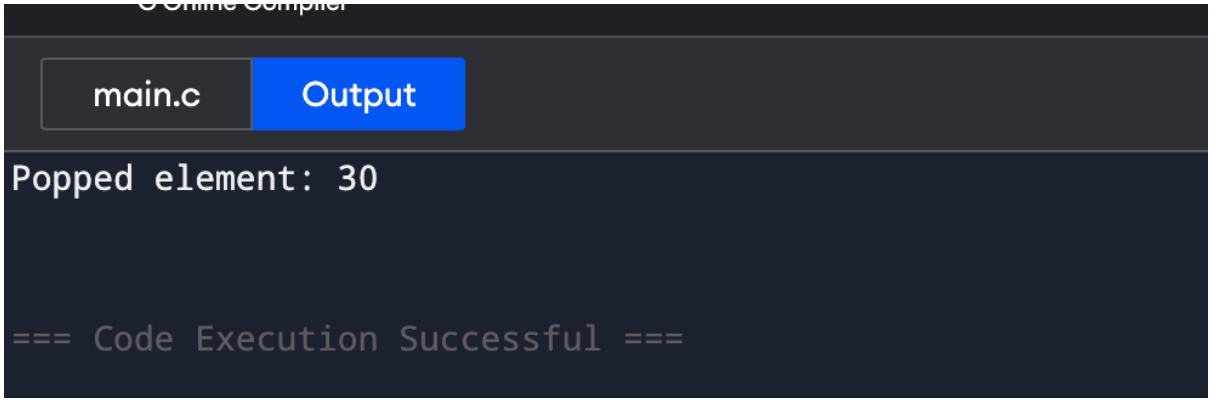


EXP NO21: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN LINKED LIST[POP]

```
// EXP21_StackPop_LinkedList.c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int val) {
    struct Node *newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = top;
    top = newnode;
}
```

```
void pop() {
    if (top == NULL)
        printf("Stack Underflow\n");
    else {
        struct Node *temp = top;
        printf("Popped element: %d\n", temp->data);
        top = top->next;
        free(temp);
    }
}

int main() {
    push(10);
    push(20);
    push(30);
    pop();
    return 0;
}
// EXP21_StackPop_LinkedList.c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;
```

```
void push(int val) {
    struct Node *newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = top;
    top = newnode;
}

void pop() {
    if (top == NULL)
        printf("Stack Underflow\n");
    else {
        struct Node *temp = top;
        printf("Popped element: %d\n", temp->data);
        top = top->next;
        free(temp);
    }
}

int main() {
    push(10);
    push(20);
    push(30);
    pop();
    return 0;
}
```



EXP NO22: WRITE A C PROGRAM TO IMPLEMENT STACK OPERATION IN LINKED LIST[DISPLAY]

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int val) {
    struct Node *newnode = (struct Node *)malloc(sizeof(struct Node));
    newnode->data = val;
    newnode->next = top;
    top = newnode;
}

void display() {
    if (top == NULL)
        printf("Stack is empty\n");
    else {
```

```
        struct Node *temp = top;
        printf("Stack elements: ");
        while (temp) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}

int main() {
    push(10);
    push(20);
    push(30);
    display();
    return 0;
}
```